AI Customer Support

AI Customer Support
Coaching Assistant

Multi-Agent AI-Assisted Training, Coaching and Performance Analytics Platform

Document Item	Details
Project Context	Infosys Virtual Internship Project - Vidzai Digital
Application	AI Customer Support Coaching Assistant
Core Stack	Python, Flask, HTML, CSS, JavaScript, Google Gemini
Interaction Modes	Simulator, Manual and Replay
Documentation Scope	Architecture, agents, orchestration, knowledge retrieval, reports, analytics, testing and difficulties
Version	Final revised technical documentation - August 2026

Prepared for technical review and final project demonstration

1. Introduction

The AI Customer Support Coaching Assistant is a web-based training platform designed to help a customer-support trainee practise realistic conversations and receive immediate feedback. Instead of behaving only as a chatbot, the application acts as a coaching environment. It analyzes the customer, recommends relevant support knowledge, evaluates the trainee response, generates communication guidance, monitors escalation risk and finally produces session-level and multi-session analytics.

The project was developed as a modular system so that individual responsibilities are separated into agents. This design makes the workflow easier to understand during a demonstration and easier to extend later. The final application supports three interaction modes: Simulator Mode for AI-generated conversations, Manual Mode for pasted customer messages, and Replay Mode for analyzing a previously completed transcript message by message.

1.1 Purpose of this Document

This document explains the implemented technical design rather than only describing the project idea. It covers the frontend and backend structure, agent responsibilities, orchestration logic, JSON communication, knowledge-base workflow, session lifecycle, reporting, analytics, validation, testing, development difficulties, current limitations and future improvements.

1.2 Main Project Output

The main output is a complete coaching experience. For each conversation turn the system can produce customer intent, sentiment and frustration information; relevant knowledge; trainee-response evaluation; strengths and improvement points; an improved response suggestion; escalation risk and reasoning; and the next customer message when applicable. At the end of the session, the same turn-level data is converted into an overall report and performance analytics.

2. Problem Statement

Customer-support training is often dependent on static scripts, mentor availability and post-conversation review. A trainee may know the basic policy but still struggle with empathy, clarity, tone, professionalism or choosing the correct next step while a customer is frustrated. Feedback given only after a session does not help the trainee understand what should have been improved at the exact moment the problem occurred.

Another difficulty is knowledge access. Support representatives may need to locate return, refund, delivery or product-specific information quickly. If the knowledge source and coaching system are separate, the trainee has to switch context. The project therefore combines conversation practice, knowledge recommendation and communication evaluation in one console.

2.1 Proposed Solution

- Generate realistic customer messages according to product, scenario, persona, difficulty and language.
- Analyze every customer message for intent, sentiment and frustration.
- Retrieve relevant guidance from uploaded knowledge documents.
- Evaluate the trainee reply using measurable communication dimensions.
- Provide AI-generated suggested responses and practical coaching feedback.
- Monitor escalation risk continuously and show the reasons behind the score.
- Store turn-level information and create a final post-interaction report.
- Track performance across multiple completed sessions to identify strengths, weaknesses and escalation patterns.

3. Objectives and Functional Requirements

1. Create a configurable support-training session.
2. Support Simulator, Manual and Replay interaction modes through a common live console.
3. Maintain conversation history so generated customers can continue naturally.

4. Separate analysis, retrieval, evaluation, coaching and escalation into modular components.
5. Use JSON structures for predictable communication between backend logic and frontend JavaScript.
6. Prevent AI-generated policy hallucination by instructing the coaching layer to use retrieved knowledge and not invent company policy.
7. Provide fallback behavior when the external AI service is unavailable.
8. Calculate turn-level and session-level scores.
9. Generate a post-interaction report with charts, strengths, improvements and customer outcome.
10. Generate performance analytics across multiple sessions.
11. Keep the code understandable enough to explain during a technical review without explaining every line.

3.1 Milestone Coverage

Milestone	Major Work Completed
Milestone 1	Base application, session configuration, simulator flow, UI and initial agent structure.
Milestone 2	Intent/sentiment analysis, knowledge recommendation, response evaluation and coaching pipeline integration.
Milestone 3	Coaching and suggested-response agent, escalation monitor, three-panel live console, Manual Mode and Replay Mode.
Milestone 4	Post-interaction summary/report, performance analytics, end-to-end testing, documentation and final-demo preparation.

4. High-Level System Architecture

The application follows a layered modular architecture. The browser contains the dashboard, session form and live console. Flask routes receive HTTP requests from JavaScript. The AgentOrchestrator coordinates the specialized agents for a complete conversation turn. SessionManager stores session state and calculated results. Knowledge routes manage uploaded support documents.

4.1 Request Flow

12. The user selects an interaction mode and creates a session.
13. The frontend sends configuration data to the Flask /create-session endpoint.
14. The backend creates a session ID and stores session metadata.
15. The live console obtains or accepts a customer message depending on the selected mode.
16. The trainee reply and current customer message are sent to /process-reply.
17. The route passes the turn to AgentOrchestrator.
18. The orchestrator calls analysis, knowledge retrieval, response evaluation, coaching and escalation components in sequence.
19. The result is returned as JSON and the live console updates its panels.
20. The processed turn is stored by SessionManager.
21. When the session ends, summary and report data are calculated.
22. Completed sessions become inputs for performance analytics.

4.2 Why an Orchestrator is Required

The orchestrator prevents the Flask route from becoming responsible for the internal logic of every agent. A route only needs to send the current session, customer message and trainee reply. AgentOrchestrator knows which components must run and in what order. If the knowledge implementation or escalation logic changes later, the route contract can remain almost the same.

4.3 How Agents Communicate

The agents do not communicate through sockets or a separate agent-to-agent protocol. They are Python classes inside the same application. Communication happens through method calls and dictionaries. For example, the sentiment result is stored in customer_analysis; the retrieved support information is stored in knowledge; both are then supplied to the CoachingAgent along with the response evaluation. Finally, the orchestrator returns all outputs in one combined dictionary.

5. Technology Stack

Layer	Technology	Use in the Project
Frontend	HTML5, CSS3, JavaScript	Pages, live interaction, DOM updates and Fetch API calls.
UI framework	Bootstrap 5	Responsive forms, alerts, buttons and layout support.
Backend	Python + Flask	Routes, validation, orchestration and server-side logic.
AI SDK	google-genai	Gemini client and AI interaction calls.
AI model	gemini-3.1-flash-lite	Used in the reviewed CoachingAgent implementation for suggestions and coaching.
Environment	python-dotenv	Loads GEMINI_API_KEY from local environment configuration.
PDF processing	PyMuPDF / fitz	Extracts text from uploaded PDF knowledge files.
DOCX processing	python-docx	Extracts paragraph text from DOCX knowledge files.
Storage	Python dictionaries + JSON/file-backed knowledge data	Prototype session and knowledge state.
Development	VS Code, Windows, virtual environment	Local coding, dependency isolation and testing.
Version control	Git/GitHub	Code collaboration and source history.

5.1 Why Flask was Suitable

Flask was suitable for the internship prototype because it is lightweight and allows routes, agent modules and frontend templates to remain easy to inspect. The project does not require a large framework to demonstrate the main AI workflow. Flask also makes JSON API endpoints straightforward to implement.

6. Project Structure and Major Files

Area	Responsibility
backend/app.py	Creates Flask application and registers application routes.
backend/routes.py	Session creation, simulator/manual processing, suggested reply, end-session and report APIs.
backend/knowledge_routes.py	Knowledge upload, list, search, delete and statistics endpoints.

Area	Responsibility
backend/session/session_config.py	Validates session configuration data.
backend/session/session_manager.py	Stores sessions/turns and calculates summaries/report data.
agents/orchestrator.py	Coordinates all agents for one complete turn.
agents/customer_simulator.py	Generates AI customer messages and maintains simulated customer continuity.
agents/sentiment_agent.py	Analyzes customer intent, sentiment and frustration.
agents/knowledge_agent.py	Provides a clean interface between orchestration and stored knowledge.
agents/knowledge_storage.py	Extracts documents, chunks text and performs current knowledge search.
agents/response_evaluator.py	Calculates communication and resolution scores.
agents/coaching_agent.py	Generates suggested replies and coaching feedback using Gemini with fallbacks.
agents/escalation_agent.py	Calculates escalation risk from analysis and message keywords.
frontend/templates	Dashboard, session configuration, simulator, report, analytics and knowledge-base pages.
frontend/static	CSS/JavaScript assets used by the UI.

7. Customer Simulator Agent

The Customer Simulator Agent represents the customer during Simulator Mode. It receives the selected product, scenario, customer persona, language and difficulty. Conversation history is also supplied so that the next message can react to the trainee's previous response instead of restarting the scenario.

7.1 Simulator Inputs

- Product, for example Amazon in the demo configuration.
- Scenario such as Return Request, Refund Delay, Damaged Product, Late Delivery or General Support.
- Persona such as Calm, Angry, Confused or Frustrated.
- Difficulty level.
- Language.
- Conversation history and session ID.

7.2 Conversation Continuity

After every processed turn, the orchestrator appends the customer message and trainee reply to conversation_history. That history is passed back to the simulator when generating the next customer message. This is the main reason a Simulator session behaves like one conversation instead of several unrelated prompts.

8. Intent, Sentiment and Frustration Analysis

The customer-analysis stage extracts information needed by later modules. Intent identifies what the customer is trying to achieve, sentiment describes the emotional direction of the message, and frustration level helps estimate escalation risk. The live console displays these values so the trainee can understand the customer before focusing only on the reply.

8.1 Why this Stage Runs Before Coaching

Coaching quality depends on customer context. A technically correct reply can still be poor if the customer is already highly frustrated and the reply ignores that emotion. Customer analysis therefore becomes an input to the CoachingAgent and EscalationAgent.

8.2 Example

For a message such as 'My refund is still pending and I have already waited for a week', the system can identify a refund-related intent, negative sentiment and increased frustration. The coaching stage can then emphasize acknowledgement, clarity and a concrete next step.

9. Knowledge Base Module

The Knowledge Base allows support material to be uploaded and retrieved during coaching. The reviewed implementation accepts PDF, DOCX and TXT files. Each document is associated with metadata such as title, product and scenario.

9.1 Upload and Processing Pipeline

23. Receive the file and metadata through the knowledge upload endpoint.
24. Validate the filename, file type and required metadata.
25. Extract text: PDF pages are read with PyMuPDF, DOCX paragraphs with python-docx and TXT as plain text.
26. Clean repeated whitespace, null characters and unnecessary blank lines.
27. Split the document into overlapping chunks so retrieval can return focused passages rather than an entire document.
28. Store document metadata and chunk data.
29. Make the processed chunks available to search and recommendation logic.

9.2 Chunking

The reviewed storage code uses character-based overlapping chunks. The default chunk size is 900 characters with an overlap of 150 characters. Overlap helps preserve context when an important sentence appears near the boundary between two chunks.

9.3 Current Search Method

The final reviewed knowledge-storage implementation uses lightweight keyword scoring. Query words are normalized and compared with words in each chunk. Matching words increase the score. Matching product and scenario metadata add additional boosts. Results are sorted by confidence and the best chunks are returned.

9.4 Embeddings and Vector Database - Current Status

Embeddings and a dedicated vector database are not claimed as active components in the reviewed final knowledge-storage code. They are a planned enhancement. An embedding model would convert chunks and queries into semantic vectors, and a vector database such as FAISS or Chroma could retrieve conceptually similar content even when exact words do not match. The current modular KnowledgeRecommendationAgent interface makes such an upgrade possible without redesigning the complete frontend.

9.5 Why this Distinction Matters

During a technical presentation, it is better to describe the implementation exactly. The project already demonstrates the important RAG-style idea of retrieving external knowledge before generating or evaluating responses, but its current retrieval mechanism is chunk-and-keyword scoring rather than a production semantic vector index.

10. Response Evaluator

ResponseEvaluator examines the trainee's submitted reply and produces measurable feedback. This component is deterministic, which makes its scoring easier to explain and prevents the entire project from depending on a generative model for every decision.

10.1 Evaluation Dimensions

- Empathy - whether the reply acknowledges the customer's concern or emotion.
- Clarity - whether the response is understandable and not unnecessarily confusing.
- Tone - whether the wording remains polite and appropriate.
- Professionalism - whether the message sounds suitable for customer support.
- Policy accuracy - whether relevant knowledge/policy guidance is respected.
- Resolution quality - whether the reply gives a useful next step or moves the issue toward resolution.
- Resolution probability - an overall indicator derived from the response dimensions.
- Conversation health - a compact description of the state of the interaction.

10.2 Strengths and Improvement Points

The evaluator does not return scores alone. It also converts score patterns into strengths and actionable improvements. This is important because a trainee needs to know what to change, not only that a score is low.

11. Coaching and Response Suggestion Agent

CoachingAgent handles natural-language coaching. It can operate before the trainee sends a response by generating a suggested support reply, and after submission by evaluating the reply in context and producing an improved response plus coaching tips.

11.1 Gemini Integration

The agent loads GEMINI_API_KEY from environment configuration and creates a Google GenAI client when the key is available. In the reviewed code, interactions are created using the model name gemini-3.1-flash-lite.

11.2 System Prompt Responsibilities

- Generate a professional support-agent reply rather than behaving like the customer.
- Evaluate empathy, tone, clarity, professionalism, policy accuracy and resolution quality.
- Use retrieved knowledge when available.
- Do not invent company policy.
- Return only valid JSON and avoid markdown code blocks.

11.3 JSON Output Conditions

The agent expects specific JSON keys. Suggested-response mode returns source, suggested_response and reasoning. Coaching mode returns source, suggested_response, strengths, improvement_tips, reasoning and a scores object. Every score is normalized to an integer between 0 and 100.

11.4 Handling Invalid AI Output

Generative models can occasionally wrap JSON in markdown code fences or return missing fields. The implementation removes optional ```json fences, parses the remaining text with json.loads(), checks object/list types, inserts defaults and normalizes scores. If parsing or the API call fails, fallback logic returns safe coaching instead of crashing the UI.

11.5 Fallback Coaching

Fallback behavior uses existing evaluation scores. For example, if empathy is below the threshold, the suggested reply emphasizes acknowledgement and concern. If clarity is weak, the fallback uses shorter and more direct sentences. This allows the application to continue functioning when Gemini is unavailable.

12. Escalation Risk Monitor

EscalationAgent is deliberately rule-based so the reason behind the alert is visible. It reads customer-analysis results and scans the customer message for escalation-related keywords.

12.1 Risk Logic

Condition	Score Added
Frustration level is High	+40
Sentiment is Negative	+25
Each configured risk keyword detected	+10

Example keywords include refund, manager, cancel, complaint, legal, angry and worst. After calculating the score, 0-39 is Low, 40-69 is Medium and 70 or above is High.

12.2 Why Rule-Based Escalation was Chosen

The trainee and reviewer can see exactly why the score changed. For a training application, this explainability is useful. A future version could combine this rule-based score with a learned risk model, but the current method is transparent and easy to validate.

13. Agent Orchestrator - Detailed Logic

AgentOrchestrator is the central coordination layer and is one of the most important files to explain during a technical review.

13.1 Initialization

When AgentOrchestrator is created, it creates objects for CustomerSimulatorAgent, IntentSentimentAgent, KnowledgeRecommendationAgent, ResponseEvaluator, EscalationAgent and CoachingAgent.

13.2 Knowledge Compatibility Helper

The _retrieve_knowledge() helper first attempts retrieve_knowledge(product, customer_message). If an older or alternate knowledge-agent implementation accepts only customer_message, a TypeError is caught and the message-only signature is attempted. Other retrieval errors are logged and an empty dictionary is returned so one knowledge failure does not stop the whole conversation.

13.3 Conversation History Helpers

_get_conversation_history() safely reads the history and guarantees a list. _store_conversation_turn() appends the latest customer message and trainee reply. These helpers keep state handling out of process_turn().

13.4 Next Customer Generation

_generate_next_customer_message() checks interaction_mode. Replay Mode returns an empty next message because replay must use the uploaded transcript. Other supported conversation flows can call the customer simulator with product, scenario, persona, language, difficulty, conversation history and session ID.

13.5 process_turn()

30. Validate session, customer message and trainee reply.
31. Analyze the customer message.
32. Retrieve relevant knowledge.
33. Evaluate the trainee reply.
34. Generate coaching using analysis, knowledge and evaluation.
35. Calculate escalation risk.
36. Store the conversation pair in history.

37. Generate the next customer message where the mode permits it.
38. Return all outputs in a single dictionary.

13.6 Why the Return is a Dictionary

A dictionary maps naturally to JSON. The Flask route can use the returned keys to build a predictable response for JavaScript. The frontend therefore does not need to know which internal class produced each result.

14. Flask Route Layer

The route layer connects browser actions with backend modules. It validates request data, loads the active session, calls the appropriate component and converts results to JSON.

Endpoint	Method	Role
/create-session	POST	Create Simulator, Manual or Replay session.
/next-turn	POST	Run a complete AI conversation turn.
/process-reply	POST	Main compatibility endpoint used by the live simulator UI.
/process-manual-message	POST	Analyze a manually entered customer message.
/generate-suggested-reply	POST	Generate AI support response before trainee submission.
/generate-message	POST	Generate a fresh AI customer message.
/analyze-message	POST	Return intent, sentiment and frustration analysis.
/end-session	POST	Complete the session and calculate final summary.
/session-report/<session_id>	GET	Return full report data.
/sessions	GET	Return stored sessions.
/run-coaching	POST	Retain the earlier coaching pipeline for compatibility.

14.1 HTTP Error Handling

The routes distinguish validation errors, missing sessions and unexpected server errors. JSON responses include a status and message so the frontend can display understandable errors rather than failing silently.

15. Session Manager and Session Lifecycle

SessionManager stores the information required to continue a conversation and later build reports. Each new session receives a UUID.

15.1 Session Data

- session_id and interaction_mode.
- product, scenario and customer_persona.
- difficulty and language.
- status, started_at and ended_at.
- turns containing processed exchanges.

- summary containing aggregated results.
- Additional runtime state such as conversation history and current customer message is attached by the route/orchestration flow.

15.2 Turn Storage

Every stored turn contains the turn number, customer message, trainee reply, customer analysis, evaluation, knowledge and timestamp. The route also attaches coaching, escalation, whether the AI suggestion was used and the suggested response shown to the trainee.

15.3 Summary Calculation

The manager averages overall score, empathy, clarity, tone, policy accuracy, resolution, professionalism and resolution probability across all turns. It also finds the highest escalation risk, reads the final conversation health and derives customer outcome and grade.

Condition	Result
Resolution probability >= 80	Likely Resolved
Resolution probability 60-79	Partially Resolved
Resolution probability < 60	Needs Escalation
Overall score >= 90	A+
80-89	A
70-79	B
60-69	C
50-59	D
Below 50	Needs Improvement

15.4 Current Persistence Limitation

SessionManager currently uses an in-memory dictionary. This is suitable for the prototype but means restarting the Flask server clears session history. A production version should store sessions and turn data in MySQL or PostgreSQL.

16. Simulator Mode

Simulator Mode is the fully generated practice flow. The first customer message is created from the session configuration. The trainee types a reply or can request an AI suggestion. When the reply is submitted, all agents process the turn and the next AI customer message is generated using the updated conversation history.

16.1 Simulator User Flow

39. Dashboard -> Simulator / New Session.
40. Choose product, scenario, persona, difficulty and language.
41. Create session.
42. AI customer sends the opening message.
43. Live panels show analysis, knowledge and escalation information.
44. Trainee enters a response or views a suggested response.
45. Submit response.
46. Evaluation and coaching panels update.
47. AI customer continues the conversation.

48. End Session creates the final report.

17. Manual Mode

Manual Mode is used when the customer message is already available. Instead of asking the AI customer to create the first message, the console displays a manual customer-message input. This is useful for classroom examples, mentor-created cases or real support-message practice.

17.1 Manual Flow

49. Create a session with interaction_mode = Manual.
50. Enter or select a sample customer message.
51. POST /process-manual-message analyzes the message, retrieves knowledge and calculates escalation risk.
52. The message becomes current_customer_message for the active session.
53. The trainee enters a reply or requests an AI suggestion.
54. /process-reply runs the common evaluation/coaching pipeline.
55. The processed turn is stored and the live summary updates.

Reusing the common pipeline is important because Simulator and Manual results remain comparable in the final report.

18. Replay Mode

Replay Mode analyzes a conversation that already happened. The implemented UI accepts a plain-text transcript containing alternating Customer: and Agent: lines.

18.1 Example Transcript Format

Customer: My refund is still pending.
Agent: I understand your concern. I will check the refund status.

Customer: I have already waited for a week.
Agent: I am sorry for the delay. Let me verify the latest update.

18.2 Replay Processing

56. Read the selected TXT file in the browser.
57. Parse Customer/User/Client labels as customer messages and Agent/Support/Representative/Trainee labels as agent messages.
58. Pair each customer message with the following agent response.
59. Display one exchange at a time.
60. Send the pair to the existing /process-reply endpoint.
61. Update analysis, evaluation, knowledge, coaching, escalation and live summary.
62. Enable Next Exchange only after the current exchange has been analyzed.
63. After the final exchange, end the session to generate the report.

18.3 Why Replay Does Not Generate a New Customer

The purpose of Replay Mode is to evaluate the historical transcript exactly as supplied. Generating a new AI customer response would change the original conversation. The orchestrator therefore returns an empty next customer message when the interaction mode is Replay.

19. Live Support Console UI

The simulator page acts as a common console for all three modes. Its central area displays the conversation and reply controls, while side panels show real-time coaching information and knowledge recommendations.

19.1 Real-Time Information

- Customer intent, sentiment and frustration.
- Relevant knowledge recommendation.
- Response-evaluation scores.
- Coaching strengths and improvement tips.
- Escalation score, level and reasons.
- Live session summary and turn count.
- Suggested agent response when requested.

19.2 Mode-Specific UI

JavaScript reads interaction_mode from sessionStorage. Simulator keeps the normal customer/agent flow. Manual shows the manual-message box. Replay disables normal reply entry and displays transcript upload, current exchange, Analyze Exchange and Next Exchange controls. This keeps one template while still giving each mode a different workflow.

20. Post-Interaction Summary and Report

When End Session is selected, SessionManager calculates the final summary and marks the session Completed. The backend returns a report URL, and the report page requests complete report data for rendering.

20.1 Report Content

- Overall score and grade.
- Customer outcome and highest escalation risk.
- Total conversation turns.
- Average empathy, clarity, tone, policy accuracy, resolution and professionalism.
- Average resolution probability.
- Turn-by-turn performance data.
- Customer frustration journey.
- Collected strengths and improvement areas.
- Personalized coaching recommendation.

20.2 Why Post-Interaction Reporting is Useful

Real-time coaching helps during the conversation, while the report helps after the conversation. It gives the trainee a single place to understand whether the interaction improved, where escalation occurred and which communication skills need further practice.

21. Performance Analytics Module

Performance Analytics aggregates completed sessions. Its purpose is to identify trends that cannot be seen from one conversation alone.

21.1 Analytics Indicators

- Completed session count.
- Average performance score.
- Improving or declining trend.
- Strongest and weakest communication areas.
- Session-by-session score trend.
- Dimension-level performance for empathy, clarity, tone, professionalism, policy accuracy and resolution.
- Common escalation triggers.
- Knowledge gaps when enough data is available.

21.2 Interpretation

For example, if empathy is consistently strong but policy accuracy remains the weakest dimension, the trainee should focus on using the knowledge panel and verifying policy before responding. If refund-related conversations repeatedly create high escalation risk, the mentor can create more refund-delay practice sessions.

22. JSON Data Contracts

JSON is the main data format between JavaScript and Flask. Consistent keys are important because a frontend/backend mismatch can display undefined values even when the backend logic is correct.

22.1 Example Complete Turn Response

```
{
  "status": "success",
  "analysis": {...},
  "knowledge": {...},
  "evaluation": {...},
  "coaching": {...},
  "escalation": {...},
  "next_customer_message": "...",
  "live_summary": {...}
}
```

22.2 Coaching JSON Conditions

- The root coaching response must be a JSON object.
- strengths and improvement_tips must be lists.
- scores must be an object.
- Every score is normalized to 0-100.
- suggested_response and reasoning are converted to clean strings.
- Invalid AI output falls back to deterministic coaching.

23. Error Handling and Reliability

23.1 Backend Validation

- Reject missing session IDs when a session is required.
- Reject empty customer messages and empty trainee replies.
- Reject attempts to add turns to completed sessions.
- Return 404 for sessions that do not exist.
- Return 400 for validation failures and 500 for unexpected processing errors.
- Return safe empty knowledge results when retrieval fails instead of stopping the whole orchestrator.

23.2 Frontend Reliability

Buttons are disabled while asynchronous operations are running, loading indicators are shown, errors are displayed in page alerts and sessionStorage is used to preserve the current session while navigating to the simulator.

23.3 AI Reliability

The project assumes that an external AI call can fail or return unexpected text. This is why CoachingAgent contains JSON cleanup and fallback behavior. This was an important implementation decision because a demo should not completely fail only because one generative response is malformed.

24. Security and Configuration Considerations

- GEMINI_API_KEY is read from environment configuration rather than hard-coded in source code.

- The .env file should be excluded from Git commits.
- Uploaded knowledge files should be restricted by file type and size in production.
- Flask debug mode should not be used for public production deployment.
- Customer transcripts may contain sensitive information, so a production version requires authentication, authorization, encryption and data-retention rules.
- Logs should avoid printing confidential customer content in a production environment.

The current project is an internship prototype, so the security architecture is intentionally lighter than an enterprise customer-support platform. These production requirements are documented as necessary future work.

25. Difficulties Faced During Development

The project involved several integration and environment issues. Recording these difficulties is important because they explain design decisions that may not be obvious from the final UI.

Difficulty	Observed Problem	How it was Handled
Virtual environment	Dependencies could be loaded from the wrong Python installation after reopening the system.	Activated the project venv before running the Flask app and verified the interpreter/path.
Google GenAI import delay	Startup sometimes appeared stuck inside google.genai, Pydantic or aiohttp imports and ended with KeyboardInterrupt when stopped manually.	Reran from the correct environment and allowed dependency initialization to finish instead of treating KeyboardInterrupt as a code bug.
SSL client initialization	Gemini client startup could pause while Python created the default SSL context.	Verified environment/network configuration; later startup completed successfully.
Frontend/backend JSON mismatch	Frontend previously read direct fields while backend returned nested analysis objects, causing undefined values.	Updated JavaScript to use the actual backend response keys.
Knowledge-agent method changes	Different versions accepted product+message or message-only parameters.	Added compatibility try/fallback logic in routes and orchestrator.
LLM JSON variability	AI output could contain markdown fences, missing fields or invalid score values.	Added output cleaning, json.loads validation, defaults, type checks and score normalization.
Mode navigation	Dashboard originally exposed Simulator but Manual and Replay were not visible as separate choices.	Added Manual/Replay navigation and query-parameter based mode preselection.
Old mode/session state	Starting a new mode could reuse old sessionStorage values and make Manual appear inside the previous Simulator state.	Cleared previous session state before storing the newly created session.
Replay implementation	Replay originally only displayed a placeholder and had no transcript-processing flow.	Implemented TXT upload, transcript parsing, exchange pairing and message-by-message reuse of /process-reply.
Report page integration	A report route without complete rendering/data wiring produced a blank/unfinished experience.	Connected SessionManager report data to the report UI and verified rendered scores/charts.
Analytics testing	Analytics requires completed sessions; empty state can look like an error.	Completed multiple test sessions and verified trends and escalation triggers.

Difficulty	Observed Problem	How it was Handled
Retrieval terminology	Project planning discussed embeddings/vector databases, but the reviewed final storage code used keyword scoring.	Documented actual implementation accurately and moved vector search to future enhancement.

25.1 Main Debugging Lesson

The most effective debugging method was to trace one complete user action across all layers: browser event -> Fetch request -> Flask route -> orchestrator -> agent output -> route JSON -> frontend DOM update. This made integration problems easier to locate than changing several files at the same time.

26. End-to-End Testing

The final milestone was tested as complete user workflows rather than only isolated functions.

Test	Expected Behavior	Status
Create Simulator session	AI opening message and active session created.	PASS
Simulator turn	All analysis/coaching panels update and next AI customer is generated.	PASS
Manual Mode	Manual message is accepted, analyzed and processed with trainee reply.	PASS
Replay Mode	TXT transcript loads, exchanges parse and Analyze/Next flow works.	PASS
Suggested Reply	AI or fallback suggested response is returned.	PASS
Escalation Monitor	Risk level and reasons update from analysis/message.	PASS
End Session	Session is completed and summary calculated.	PASS
Final Report	Overall metrics, charts, strengths and improvements render.	PASS
Performance Analytics	Multiple completed sessions produce trend information.	PASS
Mode Navigation	Simulator, Manual and Replay are accessible from the dashboard/session flow.	PASS

26.1 Replay Test Data

Replay testing used structured transcripts such as refund-delay, damaged-product and late-delivery conversations. Each Customer:/Agent: pair was processed as one exchange and the live coaching panels updated successfully.

27. Running the Project Locally

27.1 Typical Windows Commands

```
cd C:\AI\AI-Customer-Support-Coach
venv\Scripts\activate
python backend\app.py
```

After Flask starts, open the local server address displayed in the terminal, normally http://127.0.0.1:5000.

27.2 Environment Checklist

- Use the project virtual environment.
- Install project dependencies used by Flask, Google GenAI, dotenv, PyMuPDF and python-docx.
- Configure GEMINI_API_KEY.
- Run from the project root so Python imports resolve correctly.
- Keep frontend templates and static files in the paths expected by app.py.
- Use a valid knowledge-storage directory with write permission.

28. Current Limitations

- Session and analytics history is not persistent across Flask restarts because SessionManager uses in-memory storage.
- Knowledge retrieval is currently keyword/chunk scoring rather than semantic vector search.
- Replay currently expects a structured TXT transcript rather than automatically understanding every transcript format.
- The prototype does not include user login, role-based access or enterprise permissions.
- Local Flask development server is not intended for production traffic.
- Scoring rules are useful for coaching demonstration but would require calibration with real support-quality data for enterprise use.
- Large-scale document ingestion, background processing and production observability are outside the current milestone scope.

29. Future Enhancements

- Generate semantic embeddings for every knowledge chunk and query.
- Add FAISS, Chroma, Pinecone or another vector store for semantic retrieval.
- Persist users, sessions, turns, reports and analytics in MySQL/PostgreSQL.
- Add mentor and trainee accounts with role-based dashboards.
- Support CSV/JSON transcript uploads and automatic speaker recognition.
- Add richer sentiment journey visualizations and escalation trend prediction.
- Use asynchronous/background processing for large knowledge documents.
- Add automated unit tests, integration tests and continuous integration.
- Deploy behind a production WSGI server with HTTPS and managed secrets.
- Add organization-specific policy packs and configurable scoring rubrics.

30. Final Demonstration Flow

A concise technical demonstration can follow this sequence:

64. Open Dashboard and explain that the application supports Simulator, Manual and Replay.
65. Create a Simulator session and show an AI-generated customer message.
66. Submit a trainee reply and point to customer analysis, knowledge recommendation, evaluation, coaching and escalation panels.
67. Open orchestrator.py and explain the process_turn sequence instead of reading code line by line.

68. Show CoachingAgent and explain Gemini model, JSON-only prompt rules and fallback handling.
69. Show EscalationAgent and explain transparent score thresholds.
70. Show Knowledge Base and explain upload -> extraction -> chunking -> current keyword retrieval.
71. Create/mention Manual Mode and explain how a real customer message can be pasted.
72. Show Replay Mode with a TXT transcript and analyze one exchange.
73. End a session and show the post-interaction report.
74. Open Performance Analytics and explain cross-session trends.
75. Finish with limitations and future vector-database/persistent-storage improvements.

30.1 Key Technical Explanation

The most important sentence for the demo is: 'The Flask route receives one conversation turn, AgentOrchestrator coordinates the specialized agents, each agent returns structured data, SessionManager stores the processed turn, and the frontend updates the live coaching panels from the combined JSON response.'

31. Conclusion

The AI Customer Support Coaching Assistant demonstrates an end-to-end multi-agent coaching workflow. It combines AI customer simulation, customer analysis, document-based knowledge recommendation, deterministic response evaluation, Gemini-based coaching, escalation monitoring, three interaction modes, post-interaction reporting and multi-session performance analytics.

The architecture is modular and explainable. Generative AI is used where natural-language generation adds value, while deterministic validation, scoring, escalation logic and fallbacks improve reliability. The final prototype also clearly identifies its current boundaries: in-memory session persistence and keyword-based knowledge retrieval are appropriate for the internship demonstration but should be upgraded for production.

Beyond the final UI, the project provided practical experience in environment management, frontend-backend integration, JSON contracts, prompt design, handling unpredictable AI output, document processing, debugging multi-module workflows and testing complete user journeys. These lessons are directly relevant to building larger AI-assisted enterprise applications.

Appendix A. Technical Review Checklist and Expected Outputs

This appendix gives a compact verification checklist for the final project demonstration. It connects each major user action with the backend component that handles it and the visible output expected on the screen. It can also be used before a mentor review to confirm that the application is running from the correct virtual environment and that all three interaction modes still use the common coaching pipeline.

A.1 Pre-Demo Verification

- Activate the project virtual environment before starting Flask and confirm that the application imports the installed Google GenAI, Flask, dotenv, PyMuPDF and python-docx packages from that environment.
- Confirm that GEMINI_API_KEY is available through environment configuration. If the external AI service is temporarily unavailable, verify that the CoachingAgent fallback still returns a safe suggested response and coaching structure.
- Open the dashboard and verify that Simulator, Manual, Replay, Reports, Performance Analytics and Knowledge Base navigation options are visible and lead to the intended project workflow.
- Create a fresh session instead of reusing an old browser session. The newly selected interaction mode must be stored with the new session ID so that controls from an earlier mode do not appear incorrectly.
- Keep at least one processed knowledge document available when demonstrating knowledge recommendations. If no relevant document exists, explain that the system can still evaluate communication but the knowledge recommendation may be empty.

A.2 Expected Output for One Simulator Turn

After a trainee submits a reply in Simulator Mode, the browser should receive one combined JSON response. The customer-analysis area should show intent, sentiment and frustration. The knowledge panel should show relevant retrieved content when a matching chunk is available. The evaluation area should contain communication scores and feedback. Coaching should contain strengths, specific improvement tips, reasoning and an improved suggested

response. Escalation should contain a numerical risk score, Low/Medium/High level and reasons. Finally, the conversation window should receive the next AI customer message generated from the updated conversation history.

A.3 Expected Output for Manual and Replay Modes

Manual Mode begins with a customer message entered by the user rather than an AI-generated opening. Once that message is processed, the same analysis, knowledge, evaluation, coaching and escalation components are reused. Replay Mode begins with a TXT transcript. The browser parses speaker-labelled Customer and Agent lines into exchanges. Analyze Exchange processes the current pair and Next Exchange moves to the following pair. Replay must not generate a new AI customer because the purpose is to review the original historical conversation.

A.4 Expected End-of-Session Output

Ending a session changes its status to Completed and calculates the session summary. The report should show overall score, grade, customer outcome, escalation risk, average communication dimensions, journey charts, strengths and improvement areas. Performance Analytics should use completed sessions to show cross-session trends, strongest and weakest areas, improvement indicators and common escalation triggers. Because the current SessionManager is in memory, these historical sessions are available only while the same Flask process remains active.

A.5 Mentor Questions the Implementation Can Answer

- What is the final output of the project? - Real-time coaching for each support turn plus a post-interaction report and multi-session performance analytics.
- Why are there multiple agents? - Each agent has one clear responsibility, while AgentOrchestrator combines their outputs into one workflow.
- How does the frontend communicate with the backend? - JavaScript sends Fetch requests and Flask returns structured JSON.
- How is company knowledge used? - Uploaded files are converted to text, split into overlapping chunks and searched for relevant support content before coaching.
- Is a vector database currently implemented? - No. The reviewed final knowledge-storage implementation uses keyword/chunk scoring; semantic embeddings and vector storage are documented as future enhancements.
- What happens if Gemini fails? - CoachingAgent catches the error and returns deterministic fallback suggestions and feedback rather than crashing the complete session.
- How is escalation detected? - High frustration, negative sentiment and configured risk keywords add to an explainable numerical score.
- What makes Replay Mode different? - It evaluates an existing transcript exchange by exchange and does not invent a continuation to that historical conversation.

A.6 Final Readiness Criteria

The project is ready for demonstration when a fresh Simulator session can complete multiple turns, a manually entered message can be coached, a TXT transcript can be replayed exchange by exchange, an ended session produces a populated report, and at least two completed sessions can be compared in Performance Analytics. These checks verify the complete Milestone 3 and Milestone 4 flow rather than only proving that individual Python files run independently.

Appendix B. Project Quick Reference

This quick reference condenses the implemented workflow into a review-friendly form. It uses the same architecture, route responsibilities, agent roles and outputs described in the main documentation.

B.1 Core Workflow

Session configuration -> Flask route -> AgentOrchestrator -> customer analysis -> knowledge retrieval -> response evaluation -> coaching -> escalation calculation -> SessionManager -> combined JSON -> live frontend update.

B.2 Three Interaction Modes

Simulator creates an AI customer and continues from conversation history. Manual begins from a customer message entered by the user. Replay reads a structured TXT transcript, pairs Customer and Agent lines, and analyzes each historical exchange without generating a new customer continuation.

B.3 Main Agent Responsibilities

CustomerSimulatorAgent generates customer messages. IntentSentimentAgent identifies intent, sentiment and frustration. KnowledgeRecommendationAgent retrieves relevant support content. ResponseEvaluator calculates communication and resolution scores. CoachingAgent creates suggestions and coaching feedback. EscalationAgent calculates explainable risk. AgentOrchestrator combines the complete turn.

B.4 Main Visible Outputs

The live console presents customer analysis, relevant knowledge, evaluation scores, coaching strengths, improvement tips, suggested response, escalation level and reasons, live summary and turn count. Ending the session produces the post-interaction report; completed sessions feed Performance Analytics.

B.5 Reliability Decisions

The project validates JSON structures, normalizes scores, handles missing or malformed AI output, provides deterministic fallback coaching, keeps escalation rule-based for explainability and returns safe empty knowledge results when retrieval fails.

B.6 Current Prototype Boundaries

Session history is stored in memory, knowledge retrieval uses overlapping chunks with keyword scoring, Replay expects a structured TXT transcript, and the local Flask development setup does not include production authentication, persistent databases or enterprise deployment controls.

Appendix C. Technical Terms Used in the Project

These terms match the language used throughout the documentation and can be used as a compact explanation guide during a mentor review.

Agent: A Python component with one focused responsibility inside the coaching workflow.

AgentOrchestrator: The coordination layer that calls specialized agents in sequence and returns their outputs as one structured result.

SessionManager: Stores session metadata, processed turns and calculated summaries during the running Flask process.

Intent: The support goal detected from the customer message.

Sentiment: The emotional direction of the customer message used as context for coaching and escalation.

Frustration: A customer-state indicator that contributes to escalation risk.

Knowledge chunk: A focused overlapping section extracted from an uploaded PDF, DOCX or TXT document for retrieval.

Keyword scoring: The current retrieval approach that compares normalized query words with chunk content and applies metadata boosts.

RAG-style workflow: Retrieving external support knowledge before generating or evaluating coaching output.

Evaluation: Deterministic scoring of empathy, clarity, tone, professionalism, policy accuracy and resolution-related dimensions.

Fallback coaching: Safe deterministic feedback returned when Gemini is unavailable or its output cannot be validated.

Escalation risk: An explainable Low, Medium or High risk derived from frustration, sentiment and configured keywords.

JSON contract: The agreed structure of keys exchanged between Flask and frontend JavaScript.

Replay exchange: One Customer message paired with its following Agent response from the uploaded transcript.

Performance Analytics: Cross-session aggregation used to show trends, strongest and weakest areas, improvement indicators and escalation triggers.

C.1 One-Minute Architecture Explanation

The browser collects the session configuration and sends requests to Flask. Flask does not perform every coaching task itself; it passes each conversation turn to AgentOrchestrator. The orchestrator runs the specialized analysis, knowledge, evaluation, coaching and escalation components, combines their outputs into one dictionary and returns structured JSON. SessionManager stores the processed turn so the same information can later be summarized in the report and compared in Performance Analytics.

C.2 What to Show During the Final Demo

- Start with the dashboard and point out Simulator, Manual and Replay as three different entry flows into the same coaching platform.
- In Simulator Mode, submit one trainee reply and briefly identify the analysis, knowledge, evaluation, coaching and escalation panels rather than reading every value.
- Open the Knowledge Base and explain that documents are extracted, cleaned, split into overlapping chunks and searched with the current keyword-scoring implementation.
- Use Replay Mode to upload a structured TXT transcript and show that historical Customer/Agent pairs are analyzed without inventing a new customer continuation.
- End the session and show how the stored turn data becomes the post-interaction report, then open Performance Analytics to explain cross-session trends.